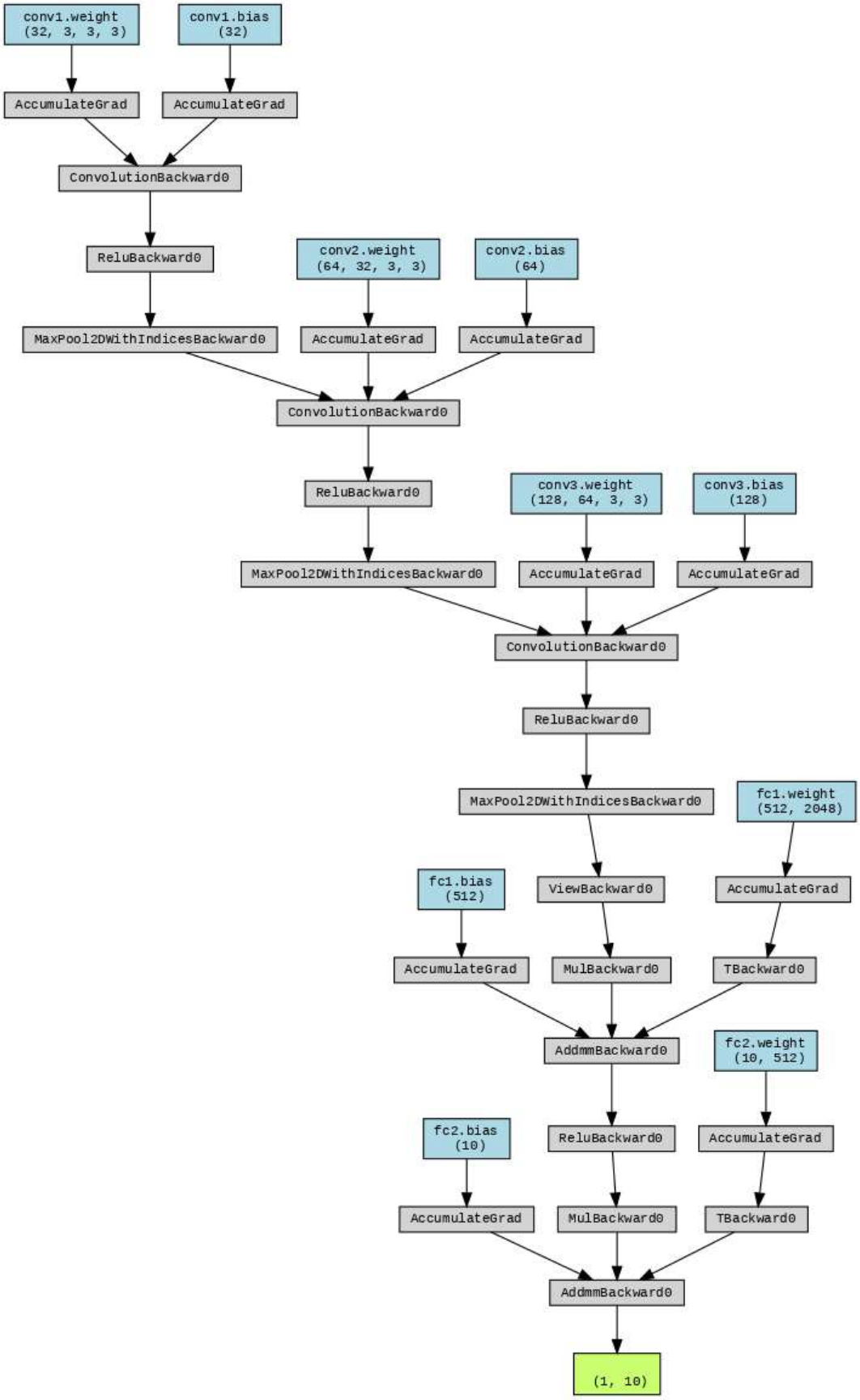
Prajwal Narayanaswamy
ID: 011839192
Email: plnu@csuchico.edu
Course: Applied Machine Learning (CSCI 611)
California State University, Chico

ASSIGNMENT-1

Image Processing Using OpenCV

Prajwal Narayanaswamy
ID: 011839192
Email: plnu@csuchico.edu
Course: Applied Machine Learning (CSCI 611)
California State University, Chico



1. Introduction

In this assignment, we applied various image processing techniques using OpenCV, focusing on convolution-based filtering. The tasks included:

- **Edge Detection using Sobel Filters**
- **Corner Detection**
- **Blurring and Scaling**
- **Chained Filtering: Blurring followed by Edge Detection**

Each technique was implemented using custom kernels, and the results were visualized through processed images.

2. Methodology

2.1 Edge Detection Using Sobel Operators

We used the Sobel operator for edge detection. Sobel filters approximate the gradient of the image intensity, highlighting areas of rapid intensity change (edges).

- Vertical Edge Detection (Sobel X):

$$K_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

- Horizontal Edge Detection (Sobel Y):

$$K_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

2.2 Corner Detection

A Convolution -like kernel was used for corner detection, which emphasizes points where edges intersect:

Prajwal Narayanaswamy
ID: 011839192
Email: plnu@csuchico.edu
Course: Applied Machine Learning (CSCI 611)
California State University, Chico

$$\text{Corner Kernel} = \begin{bmatrix} 1 & -1 & 1 \\ -1 & 4 & -1 \\ 1 & -1 & 1 \end{bmatrix}$$

2.3 Blurring and Scaling

We applied a **Gaussian Blur** to smooth the image and reduce noise. The blurred image was then down sampled using 2×2 and 4×4 subsampling.

2.4 Chained Filtering: Blurring followed by Edge Detection

To demonstrate advanced filtering, we first blurred the image and then applied Sobel edge detection to highlight major edges while reducing noise.

3. Results and Observations

3.1 Original Grayscale Image

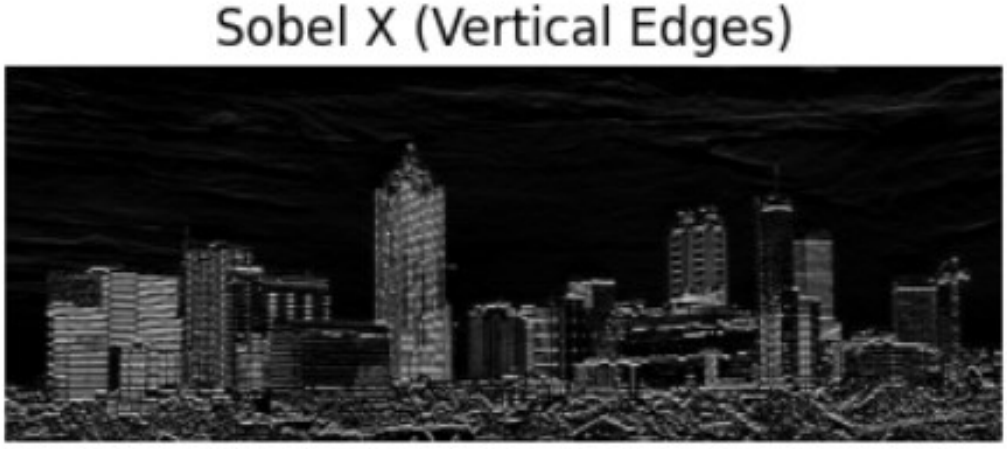
The original grayscale image provides a baseline for comparison.

Output:

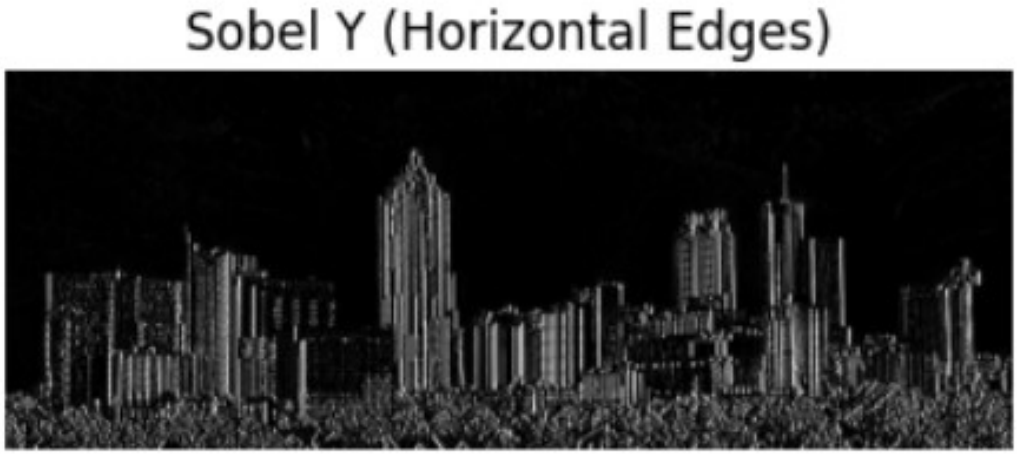
- The grayscale image shows clear intensity variations, setting the stage for edge and feature extraction.

3.2 Sobel Edge Detection

- **Sobel X (Vertical Edges):**
  Highlights vertical edges, enhancing features like walls and vertical structures in the building.



- **Sobel Y (Horizontal Edges):**
  Emphasizes horizontal edges, capturing features like rooftops and horizontal lines.



Observation:
The Sobel filters effectively detect edges in their respective directions. The

Prajwal Narayanaswamy
ID: 011839192
Email: plnu@csuchico.edu
Course: Applied Machine Learning (CSCI 611)
California State University, Chico

vertical filter accentuates vertical lines, while the horizontal filter highlights horizontal features.

3.3 Corner Detection

Output:

Highlights points where significant intensity changes occur in multiple directions, indicating corners and intersection points.

Observation:

The corner detection kernel effectively identifies key points, which are useful for object recognition and tracking.

3.4 Blurring and Scaling

- **Blurred Image:**
  The image appears smoother, with reduced noise and softened edges.

- **2×2 Scaling:**
  A downscaled version, retaining key features with reduced resolution.

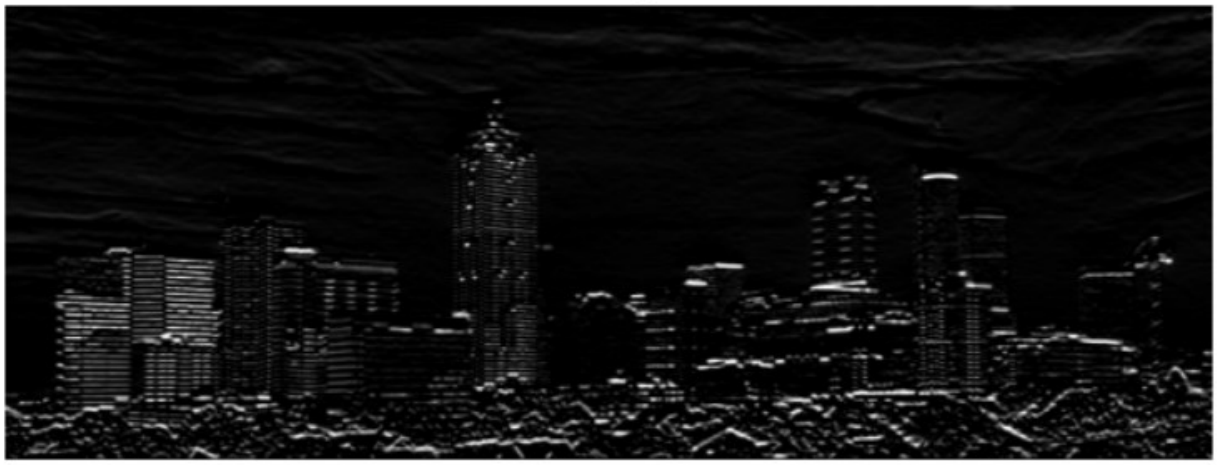
Prajwal Narayanaswamy
ID: 011839192
Email: plnu@csuchico.edu
Course: Applied Machine Learning (CSCI 611)
California State University, Chico

- **4×4 Scaling:**
  Further downscaled, showing only dominant features in lower resolution.

Observation:
Blurring effectively reduces noise, and down sampling preserves essential details while minimizing data size.

3.5 Chained Filtering: Blurring then Edge Detection

Output:
The combination of blurring and edge detection results in a clean image with prominent edges and minimal noise.



Observation:
This approach enhances major edges while suppressing minor noise-induced edges, demonstrating the benefit of pre-processing before edge detection.

4. Conclusion

The image processing tasks effectively demonstrated the power of convolutional filtering using OpenCV:

- **Sobel filters** accurately detected edges in specific directions.
- **Corner detection** identified key points in the image.
- **Blurring and scaling** reduced noise and optimized the image for further processing.
- **Chained filtering** showcased a practical approach to enhancing edge detection results.

These techniques are foundational in computer vision and are widely used in applications like object detection, image enhancement, and preprocessing for deep learning models.

Prajwal Narayanaswamy
ID: 011839192
Email: plnu@csuchico.edu
Course: Applied Machine Learning (CSCI 611)
California State University, Chico

Convolutional Neural Networks - Build Model

```
Net(
  (conv1): Conv2d(3, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (conv2): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (conv3): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (fc1): Linear(in_features=2048, out_features=512, bias=True)
  (fc2): Linear(in_features=512, out_features=10, bias=True)
  (dropout): Dropout(p=0.25, inplace=False)
)
```

Output Explanation:

The model summary shows:

- Three convolutional layers, each followed by ReLU activation and pooling.
- A pooling operation (MaxPool2d) that reduces spatial dimensions.
- Fully connected layers (Linear), transforming extracted features into class probabilities.
- A dropout layer to prevent overfitting.

This structure is a typical CNN used for small image classification tasks like CIFAR-10.

Loss Function

Implemented using torch.nn.CrossEntropyLoss(), which is suitable for multi-class classification problems.

Optimizers Tested

Two optimizers were compared:

1. **Adam Optimizer:**

- Implemented using torch.optim.Adam(model.parameters(), lr=0.001), which dynamically adapts the learning rate.
- Key parameters include betas=(0.9, 0.999), eps=1e-08, and weight_decay=0.
- Benefits: Faster convergence and better handling of sparse gradients.

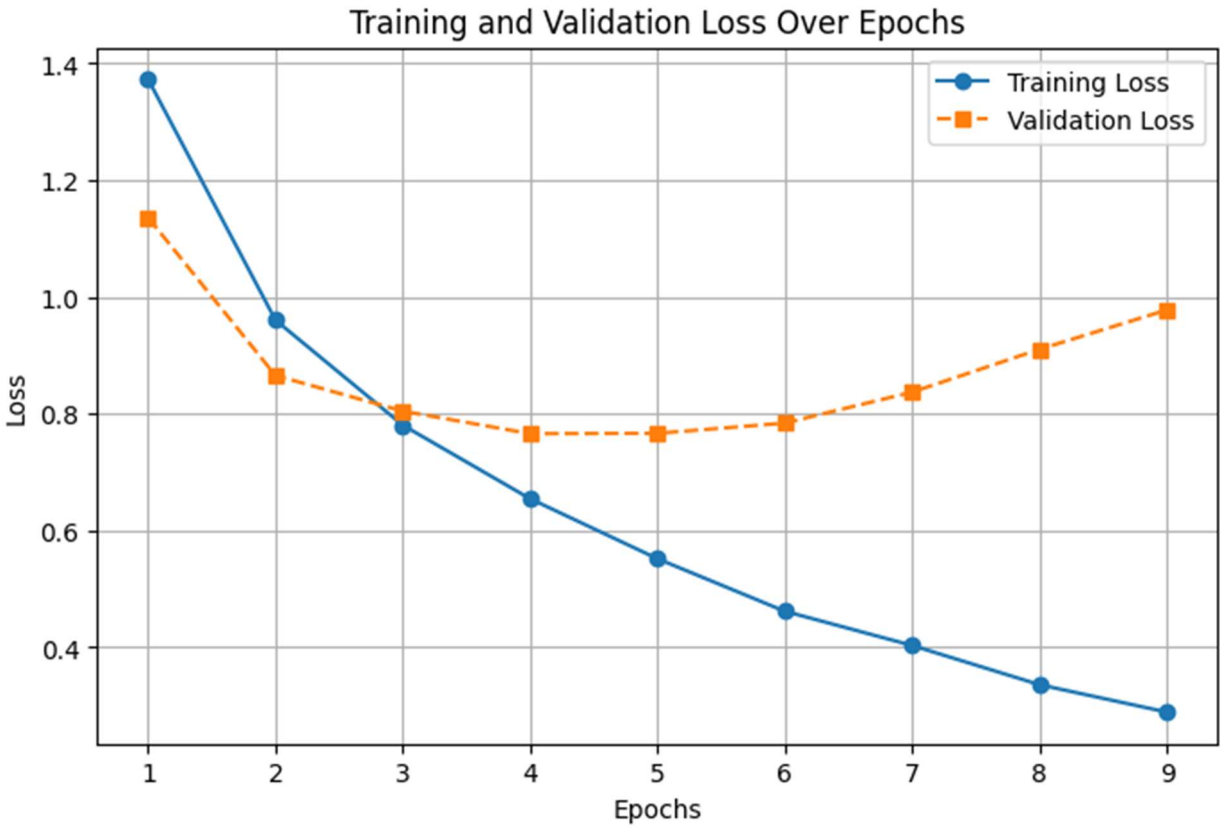
2. **SGD with Momentum:**
   - Implemented using torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9).
   - Key parameters include dampening=0, nesterov=False, and weight_decay=0.
   - Benefits: Simpler and more interpretable updates, but may require careful tuning of the learning rate.

```
Adam (
Parameter Group 0
    amsgrad: False
    betas: (0.9, 0.999)
    capturable: False
    differentiable: False
    eps: 1e-08
    foreach: None
    fused: None
    lr: 0.001
    maximize: False
    weight_decay: 0
)
SGD (
Parameter Group 0
    dampening: 0
    differentiable: False
    foreach: None
    fused: None
    lr: 0.01
    maximize: False
    momentum: 0.9
    nesterov: False
    weight_decay: 0
)
```

Prajwal Narayanaswamy
ID: 011839192
Email: plnu@csuchico.edu
Course: Applied Machine Learning (CSCI 611)
California State University, Chico

Train the Network

- Training loss continuously decreases, meaning the model is learning.
- Validation loss initially decreases, indicating improving performance.
- However, after epoch 4, validation loss stops improving and begins increasing signs of **overfitting**.
- At epoch 9, early stopping is triggered to prevent further unnecessary training

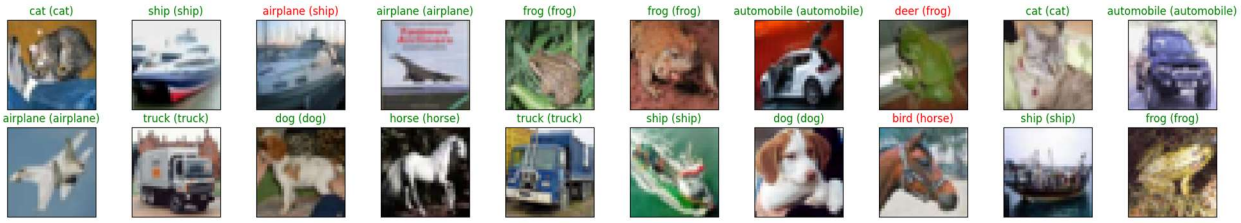


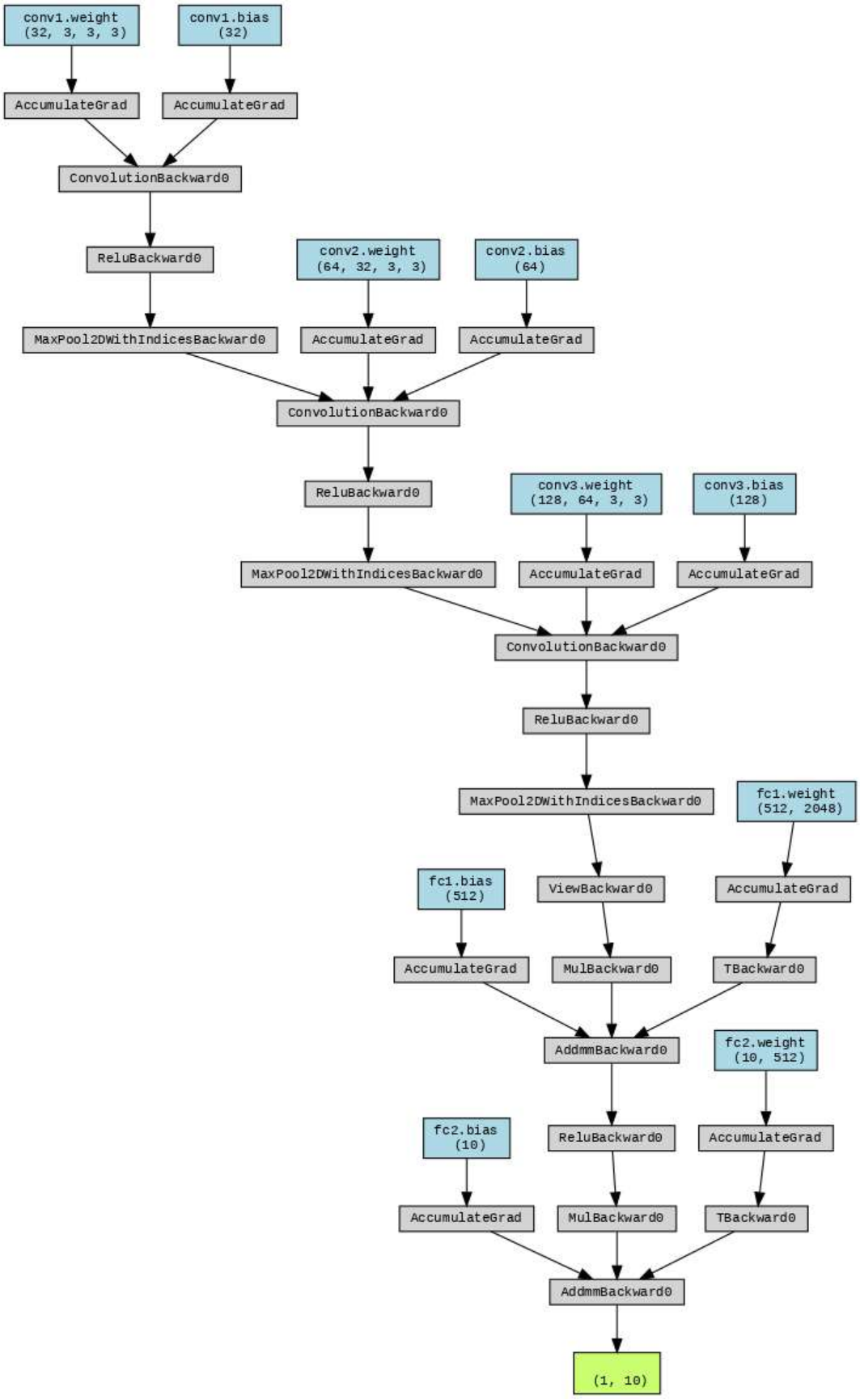
```
Test Loss: 0.790509

Test Accuracy of airplane: 79.90% (799/1000)
Test Accuracy of automobile: 91.10% (911/1000)
Test Accuracy of bird: 63.90% (639/1000)
Test Accuracy of cat: 49.30% (493/1000)
Test Accuracy of deer: 73.40% (734/1000)
Test Accuracy of dog: 69.10% (691/1000)
Test Accuracy of frog: 79.90% (799/1000)
Test Accuracy of horse: 69.40% (694/1000)
Test Accuracy of ship: 82.70% (827/1000)
Test Accuracy of truck: 72.10% (721/1000)

Test Accuracy (Overall): 73.08% (7308/10000)
```

1. **Best performing classes:**
    - **Automobile (91.10%)** and **Ship (82.70%)** were classified with high accuracy.
    - These objects have distinct features (e.g., structured edges, specific shapes), making them easier for the CNN to distinguish.
2. **Moderate performance:**
    - **Airplane (79.90%)**, **Frog (79.90%)**, **Truck (72.10%)**, and **Horse (69.40%)** had reasonably good accuracy.
    - These classes exhibit some variations but still retain distinct shapes that the model learned effectively.
3. **Lower performing classes:**
   Potential issues:
    - The model struggles with fine-grained features and overlapping patterns among similar categories (e.g., cats vs. dogs).

Prajwal Narayanaswamy
ID: 011839192
Email: plnu@csuchico.edu
Course: Applied Machine Learning (CSCI 611)
California State University, Chico

- Overfitting could be a concern since validation loss increased after a few epochs.




conv1.weight
(32, 3, 3, 3)
conv1.bias
(32)
AccumulateGrad
AccumulateGrad
ConvolutionBackward0
ReluBackward0
conv2.weight
(64, 32, 3, 3)
conv2.bias
(64)
MaxPool2DWithIndicesBackward0
AccumulateGrad
AccumulateGrad
ConvolutionBackward0
ReluBackward0
conv3.weight
(128, 64, 3, 3)
conv3.bias
(128)
MaxPool2DWithIndicesBackward0
AccumulateGrad
AccumulateGrad
ConvolutionBackward0
ReluBackward0
MaxPool2DWithIndicesBackward0
fc1.weight
(512, 2048)
fc1.bias
(512)
ViewBackward0
AccumulateGrad
AccumulateGrad
MulBackward0
TBackward0
AddmmBackward0
fc2.weight
(10, 512)
fc2.bias
(10)
ReluBackward0
AccumulateGrad
AccumulateGrad
MulBackward0
TBackward0
AddmmBackward0
(1, 10)